Candidate Ranking System

India Runs Data & AI Challenge 2026

Ranking 100K candidates against a Senior AI Engineer JD

Team: shivv23 | Solo Participant

Problem & Constraints

Goal: Rank 100K candidates against a Senior AI Engineer JD — output top-100

Scoring: 0.50×NDCG@10 + 0.30×NDCG@50 + 0.15×MAP + 0.05×P@10 — top-10 ordering is 50% of score

Phase B (ranking) must complete in ≤5 minutes on CPU, no GPU, no network calls

Honeypot rate >10% in top-100 = disqualification (~80 honeypots in dataset)

~60% of candidates (59,749) have fake/placeholder company names (Wayne Enterprises, Initech, etc.)

68.8% of candidates have non-tech titles (HR, sales, etc.) — only 10.3% have 3+ relevant ML skills

Two-Phase Pipeline Architecture

Phase A: Pre-computation (offline, ~14 min)

Load 100K candidates from candidates.jsonl

Extract 20+ features per candidate (title, skills, company, signals, education, career history descriptions)

Enrich profile_text with career_history descriptions (last 3 jobs) and education

Embed all profiles via all-MiniLM-L6-v2 (384-dim, normalized)

Build FAISS IndexFlatIP for inner-product (cosine) similarity search

Checkpoint every 5000 texts — resume after interrupt without restart

Phase B: Ranking (≤5 min CPU, no network)

Embed JD query using same sentence-transformer model

FAISS search: retrieve top-10K candidates by cosine similarity

Min-max normalize semantic scores within top-1000 for discriminability

Fuse 8 scoring dimensions (weighted linear combination)

Apply modifiers: honeypot penalty, notice period, score spread

Sort by score desc (4-decimal tie-breaking by candidate_id ascending)

Generate 3-tier reasoning (rank-dependent verbosity)

Output validated CSV: 100 rows, non-increasing scores

Scoring Dimensions (Weighted Fusion)

Dimension	Weight	Description
Semantic	20%	Profile-JD cosine similarity via all-MiniLM-L6-v2 embedding
Skills	30%	4 JD categories: AI/ML, retrieval, ranking, engineering — capped per category
Title	15%	35+ title variants with seniority-weighted scores (0.20–1.0)
Experience	10%	Bell curve centered on 7yr (target 4–10yr), penalty outside range
Signals	10%	9 behavioral signals: response rate, activity, open-to-work, GitHub, etc.
Company	5%	Product=1.0, Startup=0.6, Consulting=0.4
Location	5%	Pune/Noida or willing-to-relocate=1.0, other India=0.7, intl=0.4
Education	5%	CS/AI/ML/related fields=1.0, else=0.5
Honeypot	Modifier	Multiplicative penalty: 0–64% reduction. Not a linear weight.

Honeypot Detection (12 Rules)

1. Keyword stuffing: non-tech title + 22+ skills → +0.30
2. AI skill stuffing: non-tech title + 8+ AI keywords → +0.30
3. Impossible breadth: 22+ skills total → +0.20
4. Ghost profile: low response rate (<10%) + high profile views (>30) → +0.15
5. Inactive: 200+ days inactive + not open to work → +0.15
6. CV/speech focus without NLP/IR (JD disqualifier) → +0.20
7. All-consulting background (JD explicitly avoids) → +0.25
8. Pure research without product/startup experience → +0.20
9. Fake/placeholder company name (synthetic profiles) → +0.50
10. Non-tech keyword stuffing: 6+ AI/retrieval/ranking skills + 10+ total → +0.30
11. Tech-non-ML title with zero/≤2 AI/retrieval/ranking skills → +0.50/+0.35
12. All penalties stack to max 0.80; final: base × (1.0 − 0.8 × penalty)

Key Fixes & Design Decisions

Removed "ai" from JD_SKILLS_AI (2-char keyword caused 35,544 false-positive matches: airflow, tailwind, langchain, faiss, llamaindex all falsely counted as AI)

Enriched profile_text for FAISS embedding with career_history descriptions (last 3 jobs, 200 chars each) and education — JD query matching now sees actual work experience

Fixed empty-title substring bug ('' in key is True for every Python string, matches first TITLE_SCORES key with score 0.90)

Removed redundant additive honeypot penalty and consulting double-penalty (company_score + extra multiplier) — each signal contributes exactly once

Fake company detection: 59,749 candidates with placeholder company names (Wayne Enterprises, Initech, etc.) now penalized with +0.50

Skill relevance factor: less aggressive (0.50/0.65/0.80 instead of 0.25/0.40/0.70) to avoid double-penalizing via both skill_score and semantic_effective

Widened notice period modifier: ≤30d → +5%, >90d → -15% (was 2%/7%) — JD specifically values quick joiners

Removed 3 double-counting bugs: days_since_active, open_to_work, and PUNE_NOIDA location each contributed twice to the final score

FAISS min-max normalized over top-1000 (not top-10000) for better discriminability among candidates that could make top-100

Results

Score range: 0.5895 – 0.9704 (top-100)

Unique scores: 98/100 (2 tied pairs, resolved by candidate_id ascending)

Honeypots in top-100: 0/100 (0% — well under 10% disqualification threshold)

Fake companies in top-100: 0/100

All-consulting in top-100: 0/100

Phase B runtime: 14.2s / 300s budget (4.7% of limit)

Validation: PASS (official validate_submission.py)

Top 10 Candidates

Rank	Candidate	Company	Score	Title
1	CAND_0002025	Apple	0.9704	Senior AI Engineer
2	CAND_0079387	Microsoft	0.9378	AI Engineer
3	CAND_0087630	Vedantu	0.8483	AI Engineer
4	CAND_0045250	Rephrase.ai	0.8439	Applied ML Engineer
5	CAND_0044222	PolicyBazaar	0.8413	AI Engineer
6	CAND_0049538	Saarthi.ai	0.8365	Applied ML Engineer
7	CAND_0071974	Netflix	0.8301	Senior AI Engineer
8	CAND_0062247	Google	0.8244	AI Engineer
9	CAND_0030031	Microsoft	0.8152	AI Engineer

Tech Stack & Reproducibility

Technology

Python 3.13 — all logic, no external ranking services

sentence-transformers (all-MiniLM-L6-v2, 384-dim)

FAISS CPU (IndexFlatIP — inner product)

NumPy, pandas, scikit-learn, python-dateutil

CSV output — no hidden or manual steps

Windows 11, AMD Ryzen 7 7840U, 16 GB RAM

Reproduce

1. git clone https://github.com/shivv23/candidate-ranker-system
2. pip install -r requirements.txt
3. Place candidates.jsonl in data/raw/
4. python precompute.py (~14 min, one-time)
5. python rank.py (14s, ≤5 min budget)
6. python data/validate_submission.py data/output/submission.csv

Full validation output: PASS (format, scores, tie-breaks)

Thank You

shivv23 | Solo Participant

https://github.com/shivv23/candidate-ranker-system